Resumen ISW – Parcial 2

Qué toman en el parcial 2: práctico → diseño de casos de prueba de caja negra; teórico → diseñar un tablero Kanban.

Resaltado en amarillo: lo que remarcó la profe en clase.

1. Filosofía Lean y método Kanban (Unidad 2)

Lean

Filosofía de gestión que nace en Toyota (industria automotriz, tras la Segunda Guerra Mundial; también se la conoce como Toyota Production System) con el Just in Time, y los Poppendieck la adaptan al software (2003). Idea central: maximizar el valor para el cliente con el mínimo de recursos; todo lo que no agrega valor es desperdicio. No se queda en optimizar la línea de producción: busca satisfacer las necesidades y expectativas del cliente, optimizar el consumo de recursos, eliminar desperdicios y poner el foco donde se crea el valor, apoyándose en la capacidad de las personas para resolver problemas. Apunta a la mejora continua del proceso: no te da un proceso nuevo, sino que tomás el que ya tenés y lo mejorás aplicando sus principios. Lean no tiene valores, solo principios. Es previo a Ágil y varios principios ágiles lo heredan. Va al revés que Scrum: Lean nace en la manufactura y se adapta al software; Scrum nace en el software y después se lleva a otros ámbitos.

7 principios Lean → reglas de acción más concretas que salen de esos valores y orientan cómo trabajar

1. **Eliminar desperdicios.** Es el principio que más distingue a Lean: todo lo que no genera valor para el cliente es desperdicio y tiene que tender a cero (lo que está a medio hacer es como si no estuviera hecho). Se vincula con los principios ágiles de software funcionando y simplicidad.
2. **Amplificar el aprendizaje.** No es un aprendizaje individual: lo que uno sabe tiene que socializarse y volverse conocimiento explícito de todo el equipo (cultura de mejora continua).
3. **Embeber la integridad conceptual** (calidad incorporada desde el inicio). Que todas las partes del producto encastren con coherencia y consistencia (ligado a los requerimientos no funcionales y a pensar la arquitectura desde el principio). La calidad no se negocia: se construye paso a paso, no se deja para probar al final.
4. **Diferir compromisos hasta el último momento responsable.** Esperar da más información y menos incertidumbre (cono de la incertidumbre), pero pasado ese momento se pierde el control: no decidir también es decidir. Se relaciona con el principio ágil de decidir lo más tarde posible, pero responsablemente.
5. **Dar poder al equipo.** Equipos motivados, capacitados y capaces de autoorganizarse y tomar decisiones (en ágil, por ejemplo, el propio equipo estima su trabajo).
6. **Ver el todo.** Visión holística del producto y del valor que le da al negocio: nuestro trabajo no es generar software sino valor, y el software es la herramienta.
7. **Entregar lo antes posible.** Ciclos cortos e incrementos pequeños de valor (producto mínimo valioso, salir pronto al mercado) para retroalimentar y mejorar; se relaciona con el principio ágil de entrega frecuente.

Tipos de desperdicio: Tipo 1 = desperdicio *necesario* (se minimiza); Tipo 2 = *puro* desperdicio (se evita). Se analiza el proceso para identificar las fuentes de desperdicio y clasificarlas. El testing es desperdicio tipo 1 (necesario): en software es inevitable, y si se hace bien genera valor porque hace visible el estado del producto.

Gastos en producción Lean (7): producción en exceso, stock, pasos extra en el proceso, búsqueda de información, defectos, esperas y transportes.

7 + 1 desperdicios: son los 7 gastos de la manufactura más uno que se agregó después, el **talento no utilizado** (el +1). Lo que importa es a qué equivale cada uno en software (trabajo del conocimiento):

Manufactura	En software	Ejemplo en un equipo de desarrollo
Inventario	Trabajo parcialmente terminado	Código a medias, historias empezadas y no terminadas: no le dan valor a nadie

Manufactura	En software	Ejemplo en un equipo de desarrollo
Sobreproducción	Características extra	Funcionalidades que nadie pidió o que el usuario nunca usa
Proceso extra	Proceso extra	Documentación o aprobaciones que no le sirven a nadie
Transporte	Cambio de tareas / transferencia de conocimiento	Pasar el trabajo de un equipo a otro y que se pierda información en el traspaso
Esperas	Esperas / demoras	Una tarea bloqueada esperando la respuesta del cliente o la aprobación de otro
Movimiento	Cambio de contexto (seteo)	Saltar entre varias tareas a la vez: cada cambio cuesta volver a concentrarse
Defectos	Defectos	Bugs que obligan a rehacer trabajo
+1 Talento no utilizado	Talento no utilizado	Un dev con experiencia en arquitectura al que nadie consulta

Conclusiones: Lean pudo pasar de la fábrica al software porque los desperdicios son los mismos con otra forma. Las equivalencias que más confunden son inventario = trabajo a medias y movimiento = cambio de contexto. El trabajo a medias y la multitarea son desperdicio: por eso Kanban limita el WIP ("dejá de empezar, empezá a terminar"). Hacer de más (funcionalidades o proceso) tampoco suma si el cliente no lo valora.

Kanban: qué es

Método para **definir, gestionar y mejorar servicios** que entregan trabajo del conocimiento (servicios profesionales o actividades donde intervienen la creatividad y el diseño, de productos de software o físicos). **Kan-ban** = tarjeta de señal. Ojo con la diferencia: **Kanban** con mayúscula es el método (David J. Anderson, basado en los principios Lean); **kanban** con minúscula es la tarjeta o señal que hace visible el trabajo (ej.: el vaso de Starbucks, que lleva todos los datos del pedido). El tablero es la herramienta, no el método. **Origen (Just in Time):** a fines de los 40, Toyota (Taiichi Ohno) estudió las técnicas de almacenamiento y los tiempos de stock de los supermercados.

Valores (9): transparencia, equilibrio, colaboración, foco en el cliente, flujo (es el valor distintivo de Kanban: no se trabaja por proyectos sino con flujo continuo; el trabajo avanza lo más rápido posible, sin cuellos de botella), liderazgo, entendimiento, acuerdo, respeto.

Principios

- *De gestión del cambio:*
    - comenzar con lo que hacés ahora (entender el proceso actual, respetar roles, responsabilidades y puestos);
    - acordar la mejora a través del cambio evolutivo;
    - fomentar actos de liderazgo en todos los niveles.
- *De entrega de servicios*
    - entender y enfocarse en las necesidades y expectativas del cliente;
    - gestionar el trabajo y dejar que la gente se autoorganice en torno a él;
    - revisar periódicamente la red de servicios y sus políticas.

Prácticas generales de Kanban

Las 6 prácticas son **el qué**: lo que propone el método. Más abajo, "Cómo aplicar Kanban" es **el cómo**: los pasos concretos para armar el tablero, y cada paso baja a la práctica una de estas.

1. **Visualizar:** la herramienta es el tablero. Muestra el trabajo y su flujo y construye un modelo visual de cómo se trabaja: permite procesar mucha información en poco tiempo, habilita la cooperación (todos ven la misma imagen) y agiliza las decisiones.
2. **Limitar el trabajo en curso (WIP):** deja de empezar, empieza a terminar. Limita el trabajo en el sistema a la capacidad disponible, basándose en los datos. Se asignan límites explícitos de cuántos ítems puede haber en progreso en cada estado del

flujo de trabajo. Los sistemas eficaces se centran en el flujo y no en tener a todos ocupados: al 100% de capacidad no hay holgura y el rendimiento es mínimo. Limitar el WIP **provoca conversación y mejora**. Suele ser lo más difícil: hay que asumir que las personas no son multitasking, y sin límite por columna no se pueden evitar los cuellos de botella.

3. **Gestionar el flujo:** el flujo es el movimiento del trabajo. El objetivo es terminarlo de la forma más fluida y predecible posible, a ritmo sostenido; limitar el WIP ayuda a que el flujo sea suave y predecible. Seguir y medir el flujo da información para gestionar las expectativas de los clientes, hacer predicciones y mejorar.
    - **Administración de colas:** el foco es el flujo "fin a fin", centrado en el cliente. El proceso se mapea a un conjunto de colas de trabajo (las columnas del tablero): cada pedido pasa por ellas y cada parte del proceso le hace su aporte; se busca que no haya cuellos de botella ni estancamientos.
    - **Pull, no push:** el trabajo se toma cuando hay capacidad, no se empuja: nadie le asigna trabajo a la gente; cada uno, cuando termina, va al tablero y elige qué sigue.
4. **Hacer explícitas las políticas:** acordadas por todas las partes (clientes, interesados y quienes trabajan en el tablero) y visibles en un lugar destacado. Deben ser pocas, sencillas, bien definidas, aplicables en todo momento y fácilmente modificables por quienes prestan el servicio. No solo se hace visible el trabajo: también las formas de trabajar.
5. **Establecer ciclos de retroalimentación:** con la cadencia adecuada (reuniones periódicas); fomentan la colaboración, el aprendizaje y las mejoras, basadas en datos.
6. **Mejorar colaborativamente, evolucionar experimentalmente:** "comenzamos con lo que hacemos ahora" y "acordamos la búsqueda de la mejora a través del cambio evolutivo". Es un cambio colaborativo, impulsado por hipótesis y basado en el método científico: se diseñan experimentos en entornos donde fallar es seguro (*safe-to-fail*); si el resultado es bueno se mantiene el cambio, y si no, se vuelve fácilmente al estado anterior.

Cadencias (ejemplo de la cátedra):

Cadencia	Frecuencia (ejemplo)	Propósito
Team Kanban Meeting	Diaria	Observar y seguir el estado y flujo del trabajo (no de los trabajadores): ¿cómo entregar los elementos de trabajo más rápido en el sistema? ¿Hay capacidad disponible? ¿Qué tomar a continuación?
Team Retrospective	Quincenal o mensual	Reflexionar sobre cómo el equipo gestiona su trabajo y cómo puede mejorar
Team Replenishment Meeting	Semanal o a demanda	Seleccionar los elementos de la lista de trabajo que se van a realizar a continuación

Cómo aplicar Kanban (armar el tablero)

Es lo que se usa para diseñar el tablero en el parcial. Entre paréntesis, la práctica que aplica cada paso.

- **Proceso** (visualizar): modelarlo como columnas, con sub-columnas en progreso / hecho. Se piensa como una cadena de valor: cada parte del proceso tiene que generar valor.
- **Trabajo** (visualizar): decidir la unidad de trabajo y **dividirlo en piezas**, una tarjeta por pieza y un color por tipo de trabajo. **Tipos de trabajo:** requerimientos (casos de uso, historias de usuario, porciones de casos de uso, características), defectos (en producción y en desarrollo), desarrollo (mantenimiento, refactorización, actualización de infraestructura) y solicitudes (de cambio y sugerencias de mejora). Se les **asigna capacidad en función de la demanda** (ej.: 60% casos de uso, 30% mantenimiento, 10% defectos).
- **Límites de WIP** (limitar el WIP): son los números sobre cada columna, la cantidad máxima de piezas en esa cola. Se van ajustando a medida que el equipo aprende. Para ver quién hace qué, cada uno pone su avatar sobre la tarjeta.
- **Políticas de calidad** (hacer explícitas las políticas): reposición del tablero (cuándo, cuánto, quién), cuándo una actividad está terminada (criterios de pull: el "done" de cada estado), límites de WIP, clases de servicio, cadencia de reuniones (horarios y agendas) y otros acuerdos de colaboración.
- **Cuellos de botella y flujo** (gestionar el flujo): mover recursos al cuello de botella.
- **Clases de servicio** (políticas): distintos trabajos, distintas políticas (tabla de abajo).
- **Cadencia** (ciclos de retroalimentación): releases, planificaciones, revisiones (ver la tabla de cadencias).

Clases de servicio (ejemplo de la cátedra): definen el tiempo de respuesta que estamos dispuestos a darle al cliente. Son opcionales, pero se recomienda tener al menos una.

Clase	Color	Política
Expreso	Blanco	"Bala de plata": una urgencia que no se planifica; si aparece, se suspende todo lo demás. WIP = 1 (como máximo una en el tablero); puede exceder el límite de WIP; no se le reserva capacidad; si hace falta, se hace una entrega especial.
Fecha fija	Rosa	Fecha de entrega arriba de la tarjeta; respeta el WIP; espera en la cola hasta que convenga que ingrese; entregas programadas cuidando la fecha. Si se retrasa y peligra la fecha, pasa a expreso (el riesgo es comerse los márgenes y que termine siendo una urgencia).
Estándar	Amarillo	Lo normal, para lo que se configuró el tablero. Respeta el WIP; priorizado por valor de negocio; FIFO si no hay expresos ni fecha fija; puede analizarse por tamaño (orden de magnitud); entregas programadas.
Intangible	Verde	En software suele ser deuda técnica (migración, refactoring): trabajo interno que al cliente le cuesta ver. Mismas políticas que la estándar.

Métricas de Kanban

- **Lead Time (tiempo de entrega)** = vista del cliente(le importa a el): registra el tiempo que sucede entre el momento en el cual se pidio un ítem de trabajo y el momento de su entrega (el final del proceso). Se suele medir en días de trabajo. Mide el ritmo de entrega.
- **Cycle Time (tiempo de ciclo)** = vista interna: desde que se empieza a trabajar en la pieza de trabajo hasta que termina el proceso (empieza a contar cuando el equipo saca la tarjeta del backlog y la pone en la primera columna de actividad). Medida más mecánica de la capacidad del proceso; ritmo de terminación.
- **Touch Time:** días en que el ítem fue realmente trabajado ("tocado"), es decir en columnas de trabajo en curso, no en colas, buffers ni bloqueado.
- **Eficiencia del ciclo de proceso** = Touch Time / Elapsed Time. Relación: **Touch Time ≤ Cycle Time ≤ Lead Time**.

Métricas orientadas a servicio: no mide incremento de producto, tiene que ver con el servicio que brindo

- *expectativa* de nivel de servicio (lo que el cliente espera),
- *capacidad* de nivel de servicio (lo que el sistema puede entregar),
- *acuerdo* de nivel de servicio (lo acordado con el cliente) y
- *umbral de adecuación* (nivel por debajo del cual es inaceptable).

En Lean/Kanban las estimaciones son opcionales y pueden considerarse desperdicio. Las métricas miden el proceso y qué tan bien generamos valor con cada pieza que fluye por el tablero; se convierten en indicadores para analizar y mejorar.

2. Métricas en los 3 enfoques (Unidad 2)

Tema muy importante para la materia (posible tema de final): hay que entender qué hay detrás de cada métrica y desde qué enfoque se propone. Al explicar, poner el contexto (de qué enfoque se habla) y no mezclar.

Cada enfoque mide distinto porque gestiona distinto: el tradicional mide el plan, el ágil mide el software funcionando (producto) y Lean/Kanban mide el servicio (flujo). **La estimación predice; la métrica dice qué está pasando.** En los tres enfoques las métricas no miden la productividad de las personas: se usan para entender y mejorar.

	Tradicional	**Ágil**	**Lean / Kanban**

Proceso	Definido	Empírico	Empírico con enfoque Lean
Qué mide	El plan (avance del proyecto)	El software funcionando: mide producto	El servicio: si se cumple lo que el cliente espera
Métricas	Esfuerzo, tiempo, costos, defectos, tamaño	Velocidad, capacidad, RTF	Lead time, cycle time, touch time, eficiencia del proceso

Tradicional (procesos definidos)

- Corrección del parcial 1: es erróneo decir que en el tradicional el alcance es fijo. Del alcance se derivan tiempo y costo, pero hay desvíos, y el líder de proyecto negocia con el cliente cómo mover las variables de la triple restricción (la calidad no se negocia). Las métricas muestran cómo se mueve el plan: son clave para el monitoreo y control.

Dominios de las métricas: es el ámbito donde se aplica la métrica: no alcanza con enunciarla y que devuelva un valor, tiene que tener sentido en su dominio. Sirven para evaluar la salud del producto, del proyecto o del proceso.

- **De proyecto:** tienen razón de ser mientras dura el proyecto: cómo vamos y cómo resultó (ej.: costos actuales vs planificados).
- **De producto:** del software en sí (defectos, líneas de código, alcance).
- **De proceso:** las métricas de proyecto se consolidan para crear métricas de proceso públicas para toda la organización. El proceso se materializa en los proyectos que lo instancian: es la misma métrica, pero sobre un conjunto de proyectos. Despersonaliza y muestra cuánto se desvía el proceso definido de lo que pasa en la realidad.

Métricas básicas de un proyecto: tamaño del producto, esfuerzo, tiempo (calendario) y defectos. Tamaño y defectos son del dominio producto; esfuerzo y tiempo, de proyecto y proceso.

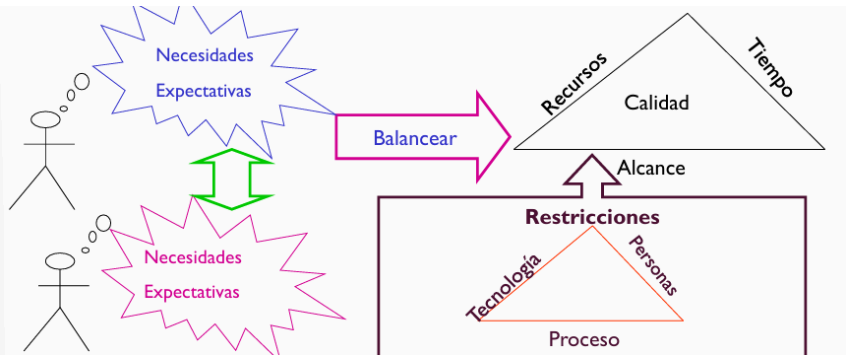
"El sueño del pibe": cada nivel quiere medir algo distinto.

- Desarrollador: esfuerzo, duración estimada vs real de tareas, % de cobertura del unit test, defectos en unit test y en revisión por pares.
- Equipo: tamaño, duración entre hitos, staffing, tareas planificadas vs completadas, distribución del esfuerzo, estado y volatilidad de requerimientos, defectos en integración/sistema y peer reviews, % de tests ejecutados.
- Organización: tiempo calendario, performance real vs planificada de esfuerzo y presupuesto, precisión de estimaciones, defectos en release.

Manténgalo simple: "si estás a millas de tu destino, no tiene sentido medir en milímetros". Las métricas posibles son muchísimas: lo que importa es quién las mira, cómo se obtienen y para qué le sirven.

- **P**reguntas: ¿nos da más información que la que tenemos?, ¿es de beneficio práctico?, ¿nos dice lo que queremos saber? Cuando tengo que ver esa métrica? → cuando todavía estés a tiempo de corregir
- Tomar la medición tiene que costar un esfuerzo mínimo y hay que decidir cada cuánto se toma. Elegirlas y observarlas es parte del rol del líder de proyecto.

Algo más sobre las métricas: sirven para **balancear** las necesidades y expectativas de los distintos interesados (que pueden chocar entre sí) contra la triple restricción: recursos, tiempo y alcance, con la **calidad en el centro** (la que no se negocia). A su vez, el alcance está condicionado por las **restricciones**: tecnología, personas y proceso.



Ágil (procesos empíricos)

- **Regla de oro:** la medición es una salida, no una actividad. Filosofía minimalista: medir lo necesario y nada más.
- **Dos principios del manifiesto que guían la elección:** "satisfacer al cliente con entregas tempranas y continuas de software valioso" y "el software funcionando es la principal medida de progreso". Por eso las métricas apuntan a cuánto software funcionando se entrega.
- Las métricas son del equipo: no se comparan ni extrapolan entre equipos.

Métrica	Qué es	Cómo se obtiene
Velocidad	Progreso del equipo: cuánto software funcionando entrega	Suma de los story points de las US terminadas (DoD) que el PO aceptó en la review. Se calcula, no se estima; las US parciales no cuentan. Corrige los errores de estimación.
Capacidad	Compromiso de trabajo para un sprint específico	Se estima en la Sprint Planning a partir de la velocidad y la disponibilidad del equipo. En los primeros sprints se calcula en horas disponibles.
RTF (Running Tested Features)	Features testeadas y funcionando	Acumulativa; se toma en la review. La curva tiene que crecer: plana o descendente es un problema. No considera la complejidad de cada feature. Está entrando en desuso porque la velocidad es más completa.

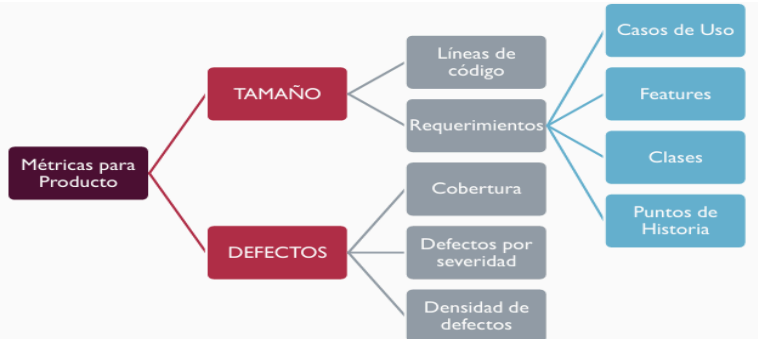
Nunca se pasa de horas a story points: un story point combina varias variables (no solo esfuerzo), así que traducirlo a horas es mezclar peras con manzanas.

Aclaración de la profe: el burndown chart no es una métrica, es una herramienta para que el equipo se gestione.

Lean / Kanban (procesos empíricos con enfoque Lean)

- Lead time, cycle time, touch time y eficiencia del ciclo de proceso (definiciones en la sección 1). En la eficiencia (touch time / elapsed time), cuanto más cerca de 1, más eficiente.
- **Métricas orientadas a servicio:** expectativa, capacidad, acuerdo y umbral de adecuación del nivel de servicio. Son propias de Kanban porque no miden el incremento de producto sino el servicio (Kanban no se aplica solo a software): si se cumple lo que el cliente espera.

Métricas de producto



Para pensar: comparar herramientas para entender, no para juzgar cuál es mejor.

3. Escalar Scrum: Nexus (Unidad 2)

No hay diapositiva de este tema entre las subidas; está armado con el resumen de Bonugli y la Guía de Nexus.

Por qué escalar: Scrum limita el tamaño del equipo (≈10 o menos), así que para productos grandes hacen falta varios equipos coordinados. Frameworks para escalar: Nexus, LeSS, SAFe, Scrum@Scale. LeSS (resumen de Bonugli): 2 a 8 equipos, un solo PO y un único Product Backlog, un Scrum Master cada 1 a 3 equipos y un sprint común; con más de 8 equipos se usa LeSS Huge.

Nexus: framework de roles, eventos, artefactos y técnicas que vincula el trabajo de **3 a 9 Scrum Teams** que trabajan sobre **un único Product Backlog**, con **un solo Product Owner**, para construir un **Incremento Integrado** "terminado" en cada sprint.

Problema que ataca: las *dependencias entre equipos*: de requerimientos (alcances que se superponen o se afectan) y de conocimiento del dominio (hay que mapear el conocimiento a los equipos para minimizar interrupciones).

Roles / responsabilidades

- **Nexus Integration Team (NIT):** responsable de que se produzca un Incremento Integrado al menos una vez por sprint; aborda restricciones técnicas y no técnicas de integración y asegura la transparencia de todos los artefactos. Ser miembro del NIT tiene prioridad sobre ser miembro de un Scrum Team; se puede estar en los dos y la composición puede cambiar con el tiempo. Lo forman el PO, un Scrum Master y uno o más miembros (a menudo de los propios Scrum Teams) que ayudan a adoptar herramientas y prácticas.
- **Product Owner:** uno solo; maximiza el valor del producto integrado y gestiona el Product Backlog.
- **Scrum Master:** asegura que Nexus se entienda y aplique; puede ser SM de uno o más Scrum Teams.

Eventos (agregan o extienden los de Scrum)

- **Nexus Sprint:** todos los equipos producen un único Incremento Integrado.
- **Refinamiento entre equipos:** continuo; descompone el PB para hacer visibles las dependencias, prever qué equipo hace qué ítem y minimizarlas.
- **Nexus Sprint Planning:** representantes de cada equipo + PO coordinan el sprint. Salidas: Nexus Sprint Goal, Sprint Goal de cada equipo, Nexus Sprint Backlog y Sprint Backlog de cada equipo.
- **Nexus Daily Scrum:** asisten *representantes* de cada equipo; se discuten problemas de integración y el progreso hacia el Nexus Sprint Goal. Las Daily de cada equipo la complementan. También identifica nuevas dependencias entre equipos, y durante el día los equipos siguen coordinando con más detalle.
- **Nexus Sprint Review:** al final del sprint, feedback sobre el Incremento Integrado. **Reemplaza** las reviews individuales de cada equipo.
- **Nexus Sprint Retrospective:** mejora de calidad y eficacia de todo el Nexus; las retros de cada equipo la complementan "de abajo hacia arriba". Inspecciona individuos, equipos, interacciones, procesos, herramientas y la DoD. La duración de cada evento de Nexus se guía por la del evento equivalente de Scrum.

Artefactos y compromisos

Artefacto	Compromiso
Product Backlog (único, del PO; debe permitir detectar dependencias)	Product Goal
Nexus Sprint Backlog (Nexus Sprint Goal + ítems de cada equipo; muestra dependencias y flujo; se actualiza durante el sprint)	Nexus Sprint Goal (suma de los Sprint Goals de los equipos)
Incremento Integrado (suma de todo el trabajo integrado del Nexus; se inspecciona en la Review pero puede entregarse antes; está terminado solo si está integrado, es de valor y es utilizable)	Definition of Done (la define el NIT; cada equipo puede usar una más estricta, nunca más laxa)

4. Retrospectivas

Reunión al final de un período de trabajo (iteración y/o release) donde el equipo mira el pasado para mejorar el futuro. Viene del principio 12 del manifiesto ("a intervalos regulares el equipo reflexiona y ajusta su comportamiento") y es el corazón del empirismo:

inspección y adaptación del *cómo* trabaja el equipo. Se analiza qué salió bien, qué no y qué mejorar, cubriendo qué hace el equipo y cómo lo hace; el cambio viene del propio equipo.

Contexto Scrum 2026 (SGEP): roles ampliados (Supporter, Stakeholder, Scrum Team: SM y PO humanos, Product Developers humanos + IA); eventos Sprint Planning, Daily, Review, Retrospective y refinamiento continuo; artefactos con compromisos (PB → Product Goal, Sprint Backlog → Sprint Goal, Incremento → DoD Output, compromiso técnico; Producto → DoD Outcome, compromiso de valor).

Ejemplo de agenda (diapo): objetivo: aprender de iteraciones previas y encontrar las causas raíz de los problemas. Agenda: inicio y descripción general, examinar la historia del proyecto, buscar patrones, analizar y sintetizar hallazgos, priorizar el plan y cierre.

Etapas (y tiempos para una retro de 2 h)

Etapa	Qué se hace	% / tiempo
1. Preparar el escenario	Objetivo, agenda, acuerdos de trabajo; preparar al equipo	5% · 6 min
2. Reunir datos	Crear una imagen compartida de lo que pasó (sin datos solo se especula)	30–50% · 40 min
3. Generar ideas	Interpretar los datos, analizar, encontrar causas y razones para el cambio	20–30% · 25 min
4. Decidir qué hacer	Propuestas de acción priorizadas (votación), experimentos, metas medibles; en iteraciones cortas no más de 2–3 cosas en *dejar de hacer / seguir haciendo / empezar a hacer*. Si no se hace, la retro queda trunca	15–20% · 12 min
5. Cerrar	Agradecer, repasar las mejoras, quién implementa qué, cierre	10% · 17 min

Cuándo "huelen mal"

Respuestas que vienen de los líderes; participan pocos; nadie habla del "elefante" (o se lo disfraza); se busca la bala de plata; se hacen donde trabaja el equipo; duran 20 min o menos; no se priorizan mejoras; no se implementaron las anteriores; no hay preparación.

Para recordar

- No son para dar feedback de performance individual.
- Los gerentes no asisten salvo invitación del equipo, y solo observan.
- No reemplazan la falta de excelencia técnica o de habilidades; no arreglan todo.
- La retrospectiva adapta el proceso, no el producto: no hay trazabilidad entre la retro y el incremento (aclaración de la profe sobre el parcial 1).
- Mirar solo lo que salió bien lleva a errores serios; media hora no alcanza; cada retro es única: elegir dónde hacer foco y actividades que hagan participar activamente a todos.
- Sin retrospectivas no hay mejora continua: un equipo que no las hace compromete su sustentabilidad y la calidad del producto.
- Ayudan a los equipos a pensar, decidir y actuar juntos, a reevaluar sus diseños y a cambiar su cultura; con el tiempo, hacen que buenos equipos se conviertan en grandes equipos.

5. Prácticas continuas (Unidad 3)

No hay diapositiva de este tema entre las subidas; está armado con material complementario (clase y resúmenes de compañeros).

Son la forma en que SCM se aplica en ágil: automatizar para integrar, probar y entregar seguido y con poco riesgo. **La diferencia entre las tres es cuánto se automatiza** (pruebas y pase a producción).

Práctica	Qué es	Clave
Integración continua (CI)	Integrar el código al repositorio compartido varias veces al día; cada integración se verifica con build + pruebas automatizadas (unitarias, idealmente con TDD)	Detecta problemas temprano; el código siempre compila y pasa las pruebas
Entrega continua (Continuous Delivery)	El software está *siempre listo* para pasar a producción; se suman pruebas de aceptación automatizadas	El pase a producción es una **decisión humana** (el PO). CI es prerrequisito
Despliegue continuo (Continuous Deployment)	Pasa a producción **automáticamente** mediante pipelines	Sin intervención humana

Estrategias de deployment (buscan que el cambio sea transparente para el usuario, sin downtime y con reversión fácil):

- **Canary:** se libera primero a una porción chica de usuarios, se observa y después se extiende al resto. Las fallas tienen impacto limitado.
- **Blue/Green:** dos entornos de producción casi idénticos; el tráfico pasa del azul (versión vieja) al verde (nueva). El azul queda en espera para revertir.

SCM en ágil (contexto): sirve al equipo y no al revés; coordina en vez de controlar; responde a los cambios; automatiza todo lo posible; es responsabilidad de todo el equipo; tareas de SCM embebidas en el sprint; no hay comité de cambios (prioriza el PO) y las auditorías pueden obviarse.

6. Calidad: conceptos generales y calidad de producto (Unidad 4)

Calidad: todos los aspectos y características de un producto o servicio que se relacionan con su habilidad de alcanzar las necesidades **manifiestas o implícitas**. Es relativa: depende de las personas, sus circunstancias y el tiempo (las percepciones cambian).

Situación de los proyectos (por qué importa): ~90% retrasados, 50% exceden presupuesto, 50% no alcanzan los objetivos, 30% cancelados. Síntomas típicos: atrasos, costos excedidos, falta de cumplimiento de los compromisos, requerimientos poco claros, el software no hace lo que debe, horas extra, fenómeno del 90-90, "¿dónde está ese componente?".

Un software de calidad satisface: las expectativas del cliente y del usuario, las necesidades de la gerencia, las del equipo de desarrollo y mantenimiento, y de otros interesados.

Principios de calidad

- La calidad no se "inyecta" ni se compra: debe estar **embebida** (no se agrega al final).
- Es un esfuerzo de todos; las personas son la clave (capacitación).
- Necesita sponsor gerencial, pero se puede empezar por uno; liderar con el ejemplo.
- No se puede controlar lo que no se mide.
- Simplicidad: empezar por lo básico.
- El aseguramiento de calidad debe **planificarse**.
- **Aumentar las pruebas no aumenta la calidad.**
- Debe ser razonable para mi negocio.

Visiones de la calidad (¿calidad para quién?)

Visión	Calidad es…
Trascendental	Algo que se reconoce pero no se puede definir
Del usuario	Adecuación al uso: satisface lo que el usuario necesita
De manufactura	Conformidad con la especificación / requerimientos

Visión	Calidad es…
Del producto	Atributos internos medibles del producto
Basada en el valor	Lo que el cliente está dispuesto a pagar por ella

Gestión de la calidad: buscar la coincidencia entre la calidad necesaria (lo mínimo que el cliente necesita), la programada (lo planificado) y la realizada (lo construido); la gestión de la calidad acerca los tres círculos para que coincidan con la **calidad esperada**. Todo lo que quede fuera de esa coincidencia es desperdicio o insatisfacción.

Dónde se trabaja la calidad en el software

- **Proceso, proyecto y producto.** Hay modelos para crear, mejorar, evaluar y medir procesos/productos.
- **Técnicas de aseguramiento de calidad** (preventivas, durante el proceso): revisiones técnicas y auditorías (de configuración).
- **Técnicas de control de calidad** (sobre el producto): testing (y también revisiones y auditorías).

Modelos de calidad de producto

ISO/IEC 25000 (SQuaRE): familia que incluye calidad de producto y en uso (ISO 25010:2011) y calidad de datos (ISO 25012:2008).

ISO 25010 – 8 características de calidad de producto (con subcaracterísticas):

Característica	Subcaracterísticas
Adecuación funcional	Completitud, corrección, pertinencia funcional
Eficiencia de desempeño	Comportamiento temporal, utilización de recursos, capacidad
Compatibilidad	Coexistencia, interoperabilidad
Usabilidad	Reconocimiento de adecuación (en la diapo: "de corrección"), aprendizaje, operabilidad, protección frente a errores de usuario, estética, accesibilidad
Fiabilidad	Madurez, disponibilidad, tolerancia a fallos, capacidad de recuperación
Seguridad	Confidencialidad, integridad, no repudio, autenticidad, responsabilidad
Mantenibilidad	Modularidad, reusabilidad, analizabilidad, capacidad de ser modificado, capacidad de ser probado
Portabilidad	Adaptabilidad, facilidad de instalación, capacidad de ser reemplazado

McCall: organiza la calidad en 3 ejes. *Operación del producto:* corrección, fiabilidad, usabilidad, integridad, eficiencia. *Revisión del producto:* facilidad de mantenimiento, flexibilidad, facilidad de prueba. *Transición del producto:* portabilidad, reusabilidad, interoperatividad.

Barbacci / SEI: atributos como performance, seguridad y confiabilidad; distingue el óptimo *local* (un solo atributo) del óptimo *global* (varios atributos a la vez, que compiten entre sí).

Trabajar para y con calidad: ventajas y desventajas

Las diapositivas no las listan como tales; armado a partir de sus puntos clave.

- **Ventajas:** trabajar con calidad es más barato que sin ella (menos retrabajo, que es 40–50% del desarrollo); defectos detectados temprano cuestan mucho menos; más predictibilidad en plazos y costos; clientes satisfechos y crecimiento del negocio; la mejora de procesos demostró retornos de inversión sustanciales.

- **Desventajas / costos:** inversión inicial (capacitación, infraestructura, herramientas, definición de procesos); al principio parece ralentizar el desarrollo; requiere compromiso y cambio organizacional; mal aplicada puede volverse burocracia (usar prácticas inapropiadas solo porque son estándar).

7. Calidad de proceso, aseguramiento de calidad y modelos (CMMI – SPICE – ISO)

El proceso es el único factor *controlable* para mejorar la calidad del software y el rendimiento de la organización (los otros —personas, tecnología, cliente, negocio, entorno— no se controlan igual). Por eso los procesos definidos asumen que la calidad del producto depende de la calidad del proceso.

- **Proceso (IEEE):** secuencia de pasos ejecutados para un propósito dado.
- **Proceso de software (Sw-CMM):** conjunto de actividades, métodos, prácticas y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados. Combina procedimientos y métodos + herramientas y equipos + personas con habilidades, entrenamiento y motivación.
- Definirlo = responder quiénes participan, qué hacen, cuándo y cómo. Lo que sirve a unos puede no servir a otros.
- **Proceso para construir software (diapo):** etapas de ingeniería, implementación, prueba y despliegue, atravesadas por actividades de soporte: planificación y seguimiento de proyectos, administración de configuraciones y aseguramiento de la calidad.
- "Lo que no está controlado no está hecho" (Watts Humphrey).

Administración de la calidad del software

Asegura que se alcancen los niveles de calidad requeridos: define estándares y procesos apropiados, asegura que se respeten y busca una **cultura de calidad** (responsabilidad de todos).

Actividad	Qué hace
Aseguramiento de calidad	Establece estándares y procedimientos *organizacionales* de calidad
Planificación de calidad	Selecciona (y adapta) los estándares y procedimientos aplicables a *un proyecto*; produce el **plan de calidad** (productos de calidad deseados, cómo se evalúan, atributos más significativos)
Control de calidad	Asegura que el equipo respete esos estándares y procedimientos. Dos enfoques: **revisiones de calidad** y **evaluaciones automáticas / mediciones**

Revisiones de calidad (principal método de validación de proceso o producto): un grupo examina parte del proceso o producto y su documentación. Tipos: inspecciones para remover defectos (producto), revisiones de progreso (producto y proceso), revisiones de calidad (producto y estándares).

Funciones del aseguramiento de calidad: desarrollar herramientas, técnicas y estándares para las revisiones; evaluar la planificación del proyecto, los requerimientos, el proceso de diseño, las prácticas de programación, la integración y prueba, y la planificación y control de proyectos; adaptar los procedimientos de QA a cada proyecto.

Grupo de Aseguramiento de Calidad (GAC): no debe reportar al líder de proyecto (independencia); no más de una posición entre la gerencia de primer nivel y el GAC; reportar a alguien realmente interesado en la calidad.

Estándares: clave de la gestión de calidad efectiva; internacionales, nacionales, organizacionales o de proyecto. *De producto:* características que deben tener los componentes (ej. estilo de programación). *De proceso:* cómo deben implementarse los procesos. Ojo: no usar prácticas inapropiadas solo porque son estándar.

Ciclo de procesos basados en calidad: definir proceso → desarrollar producto → evaluar calidad → ¿OK? sí: estandarizar; no: mejorar el proceso y repetir.

La administración de calidad y el desarrollo (diapo): el proceso de administración de calidad acompaña al proceso de desarrollo y le aporta estándares y procedimientos, el plan de calidad e informes de revisión de calidad.

Calidad de procesos en la práctica: definir procesos estándar (cómo conducir las revisiones, cómo hacer la administración de configuración, etc.), monitorear el desarrollo para asegurar que se respeten y reportar a la administración del proyecto y al responsable del software.

Modelos de mejora de procesos

Los modelos son **descriptivos**: dicen *qué* lograr, no *cómo*. Un proyecto de mejora tiene como resultado un *proceso mejorado*, no un producto.

IDEAL (ciclo de 5 fases para un proyecto de mejora):

1. **Initiating (Iniciar):** estímulo para mejorar, fijar contexto y conseguir **sponsor**, establecer la infraestructura de mejora.
2. **Diagnosing (Diagnosticar):** evaluar y caracterizar la práctica actual (dónde estamos vs dónde queremos ir: análisis de brecha), desarrollar recomendaciones.
3. **Establishing (Establecer):** fijar prioridades y estrategias, planificar acciones, formar equipos de acción.
4. **Acting (Actuar):** definir procesos y métricas, planificar y ejecutar **pilotos**, luego instalar en la organización.
5. **Learning (Aprender; en la lista de la diapo figura como Leveraging):** documentar y analizar lecciones, revisar el enfoque organizacional para el próximo ciclo.

SPICE (Software Process Improvement Capability Evaluation, ISO/IEC 15504): modelo para evaluar y mejorar la capacidad de los procesos; clasifica en 6 niveles (0 a 5). Es una norma ISO.

CMMI (Capability Maturity Model Integration)

- Evolución del SW-CMM; lo emite el **SEI** (fundado en 1984 por el DoD de EE.UU. y Carnegie Mellon). v1.3 (2010); desde 2020 lo maneja **ISACA**, versión vigente **2.0**.
- Uno de los modelos más implementados. **No es una norma y no se certifica**: se *evalúa* (appraisal) por Lead Appraisers reconocidos.
- **Constelaciones:** CMMI-DEV (desarrollo), CMMI-SVC (servicios), CMMI-ACQ (adquisición); comparten 16 áreas de proceso fundacionales (el 77% de la inversión es potencialmente reusable).

Representaciones

	Por etapas	Continua
Niveles	5 (1 a 5) de **madurez organizacional**	6 (0 a 5) de **capacidad** por área de proceso
Definidos por	Un conjunto de áreas de proceso	Cada área de proceso
Ventaja	Una única clasificación comparable entre organizaciones; secuencia de mejora probada	Elegir el orden de las mejoras; comparar por área

Niveles de madurez (por etapas) y áreas de proceso (v1.3)

Nivel	Característica	Áreas de proceso
1 Inicial	Procesos impredecibles, pobremente controlados, reactivos	—
2 Administrado	Procesos por proyecto, a menudo reactivos	REQM (Adm. de requerimientos), PP (Planificación), PMC (Monitoreo y control), SAM (Acuerdo con proveedores), MA (Medición y análisis), **PPQA (Aseguramiento de calidad de proceso y producto → auditorías, sección 9)**, CM (Adm. de configuración → prácticas continuas, sección 5)

Nivel	Característica	Áreas de proceso
3 Definido	Procesos de la organización, proactivos	RD, TS, PI, **VER** (Verificación), **VAL (Validación) (→ revisiones y testing, secciones 8 y 10)**, OPF, OPD, OT, IPM, RSKM (Riesgos), DAR
4 Cuantitativamente administrado	Procesos medidos y controlados	QPM, OPP
5 Optimizado	Foco en la mejora continua	CAR (Análisis causal), OPM

A mayor nivel: más productividad y calidad, menos riesgo. En CMMI lo importante es que *alguien* cubra cada rol o grupo (según el tamaño de la organización).

ISO (en la materia): ISO 25010 evalúa calidad de *producto*; ISO 15504 (SPICE) evalúa *procesos*; ISO 9001 es una norma de gestión de calidad *certificable* (a diferencia de CMMI).

Puntos clave (diapo): el software puede analizarse como proceso y como producto, y la calidad también; la calidad del software es difícil de medir; el software como proceso es el fundamento para mejorar la calidad; trabajar con calidad es más barato que sin ella; la mejora de procesos exitosa requiere compromiso y cambio organizacional, hay varios modelos para darle soporte y demostró retornos de inversión sustanciales.

CMMI vs Ágil

	CMMI	Ágil
Valor esencial	Medir y mejorar el proceso (mejor proceso → mejor producto)	Respuesta al cliente, mínima sobrecarga; refinamiento de requerimientos con metáforas y casos de negocio
Personas	Disciplinadas, siguen reglas, aversión al riesgo	Cómodas, creativas, toman riesgos
Comunicación / conocimiento	Organizacional (macro); el conocimiento está en los activos de proceso	Persona a persona (micro); el conocimiento está en las personas
Éxito por	Predictibilidad, estabilidad	Aprovechar oportunidades, velocidad, performance
Confianza del cliente	En la infraestructura de proceso	En el software funcionando y en los participantes
Enfoque	Descriptivo, cuantitativo, universal, estratégico, foco en actividades, riesgo proactivo	Prescriptivo, cualitativo, situacional, táctico, foco en producto ("sólo hazlo"), riesgo reactivo
Calidad	Cargado al frente (mover a la derecha)	Conducido por pruebas (mover a la izquierda)
Mejora	Organizacional: uniformidad, nivelación	En el proyecto: tradición oral, innovación
Cuerpo de conocimiento	Cruza dimensiones, estandarizado	Personal, evoluciona, temporal
Reglas de atajo	Desalentadas	Alentadas
Quién decide	Comités	Individuos

	CMMI	Ágil
Alcance de la vista (involucrados, producto)	Amplio, inclusivo, organizacional	Pequeño, focalizado
Nivel de discusión	Palabras y definiciones; duradero, exhaustivo	El trabajo en mano
Foco de negocio	Interno, reglas	Externo, innovación

Similitudes: ambos buscan organizaciones de alto desempeño, ambos planifican, ambos tienen reglas, ninguno es completo ni aplicable a cualquier proyecto, ambos se basan en experiencia (no son ideas nuevas); y, en broma, ambos son "CMMs" (Consultant Money Makers). Las reglas son requerimientos del proceso: violarlas tiene consecuencias serias (en CMMI, el SEPG y la política de proceso). Hipótesis de la diapo: CMMI tolera Ágil (hay áreas de proceso que lo soportan, otras neutrales y otras en conflicto, y se puede evaluar en un ambiente ágil); la pregunta inversa es si Ágil deja la puerta abierta a CMMI. Lo resume como caos y creatividad (Ágil) frente a ley y disciplina (CMMI). ¿Se pueden unir? **(diapo "CMMI cara a cara con Ágil")** Marca las prácticas de cada nivel en **verde** = Ágil las soporta, **negro** = neutral, **rojo** = desigual. Nivel 1 (identificar el alcance, realizar el trabajo): verde. Nivel 2: verde requerimientos/objetivos/planes, recursos adecuados, asignar responsabilidad y autoridad, capacitar y administración de configuración; neutrales la política organizacional, identificar involucrados, monitorear y controlar el plan y revisar con la gerencia; rojo monitorear objetivamente la adherencia a los procesos y el QA de productos. Nivel 3: mantener un proceso definido, neutral; medir la performance del proceso, rojo. Nivel 4: rojo (objetivos cuantitativos, estabilizar subprocesos). Nivel 5: mejora continua, verde; corregir la causa raíz, neutral. O sea, el choque está en lo cuantitativo y en el control objetivo del proceso.

8. Revisiones técnicas

Las revisiones técnicas son el proceso de V&V **estático** (no ejecutan el programa) cuyo objetivo es detectar defectos y corregirlos en etapas tempranas. Complementan al testing (dinámico).

Verificación y validación

- **Verificación:** ¿estamos construyendo el producto *correctamente*? (corrección: lo construido respeta la especificación).
- **Validación:** ¿estamos construyendo el producto *correcto*? (correspondencia con el mundo real y la necesidad del cliente).
- Es un proceso de **ciclo de vida completo**: empieza con revisiones de requerimientos, sigue con diseño, inspecciones de código y prueba. Debe **planificarse**, es caro y conviene empezar temprano: el costo de reparar una falla crece ×10 a ×100 de requerimientos a release.
- Dos aproximaciones complementarias: **revisiones técnicas** y **pruebas de software**.

Por qué existen las fallas: ruido de comunicación (con n personas hay n(n-1)/2 caminos: 3 → 3, 5 → 10, 10 → 45) y límites de la memoria de corto plazo (7 ± 2). Las fallas más persistentes vienen de la complejidad inherente del producto.

Retrabajo: el retrabajo evitable es el 40–50% del desarrollo.

Error, defecto y falla (vocabulario de revisiones)

- **Falla:** error en un producto de trabajo (cualquier salida de una actividad del ciclo de vida).
- **Error:** falla originada en el producto de trabajo *que se está inspeccionando*.
- **Defecto:** falla originada en un producto de trabajo *predecesor* (se arrastra de una etapa anterior).

Severidad	Qué es	Ejemplos
Mayor	Puede causar una falla operacional o un resultado inesperado	Error lógico o estructural en código; expresión de diseño que causaría una falla si se implementara tal cual; requerimiento ambiguo o con información faltante; caso de prueba que no detectaría fallas o no se puede repetir
Menor	Condición no deseable que no causa falla operacional	Violación de estándares de codificación; requerimiento que no se puede probar; información poco clara o

Severidad	Qué es	Ejemplos
		redundante en el plan o los casos de prueba (genera testing innecesario)
Cosmético	Tipeo, ortografía; no se cuenta como falla	Errores de gramática, plantilla o historial desactualizado, copyright; una sugerencia alternativa (ej.: otro algoritmo de búsqueda)

Principios de V&V: la prevención es mejor que la cura; evitar es más efectivo que eliminar; la retroalimentación enseña; priorizar lo rentable; olvidarse de la perfección; enseñar a pescar.

Características de las revisiones

- Origen: **Fagan (1976)**, inspecciones basadas en experiencia de hardware, para detectar defectos lo más cerca posible de donde se generan. Se practican en industrias donde la calidad y el retrabajo son críticos, y hay muchas variantes de las inspecciones de Fagan.
- Se puede revisar **cualquier representación legible** del software, en cualquier momento del desarrollo, incluso versiones incompletas.
- "El trabajo técnico necesita ser revisado por la misma razón que los lápices necesitan gomas": a quien origina un error se le pasa más fácil que a otros. Además motiva a hacer mejor el trabajo.
- **Ventajas:** descubren muchos errores; sirven sobre versiones incompletas; consideran otros atributos de calidad.
- **Desventajas:** difíciles de introducir formalmente; cargan costos al inicio (el ahorro llega cuando el equipo gana experiencia); requieren tiempo y parecen ralentizar.
- **Costos:** de infraestructura (entrenamiento, plantillas, guías, programa de medición, herramientas), operacionales (tiempo individual y grupal, informes) y adicionales (preparar material, calendario, datos).

Tipos: walkthrough vs inspección

	Walkthrough (recorrida)	Inspección
Objetivo	Mínima sobrecarga, capacitar desarrolladores, retorno rápido	Detectar y remover todos los defectos eficiente y efectivamente
Atributos	Informal, poca o ninguna preparación, sin mediciones (no es FTR)	Formal, checklists, mediciones, fase de verificación
Cómo	Un diseñador o programador (el autor) guía al equipo y a otros interesados por el producto; los participantes preguntan y comentan sobre posibles errores, violaciones de estándares y otros problemas	Reunión planificada con roles definidos
Resultado	Buenos resultados, pocas métricas, sin control del proceso	Mejores resultados, proceso controlado, métricas útiles en todo el ciclo

Revisores sugeridos por documento: arquitectura (arquitecto, analista, diseñador, líder, testers); diseño detallado (diseñador, arquitecto, programadores, testers); plan de proyecto (líder, stakeholders, ventas/marketing, líder técnico, calidad); ERS (analista, líder, arquitecto, diseñador, testers, ventas); código (programador, diseñador, testers, analista); plan de testing (tester, programador, arquitecto, diseñador, calidad, analista).

En resumen (diapo): solo mejorar las pruebas es caro y no sirve para remover errores en etapas tempranas; los walkthroughs dan buenos resultados, pero con pocas métricas y sin control del proceso; las inspecciones dan mejores resultados, con el proceso controlado y métricas útiles en todo el ciclo de vida.

Inspección

Actividad de garantía de calidad. **Objetivos:** descubrir errores, verificar que se cumplen los requisitos, garantizar que se respetan los estándares, desarrollo uniforme, proyectos más manejables.

- **Son:** la forma más barata y efectiva de encontrar fallas, fuente de métricas, conocimiento cruzado, trabajo en grupo, método probado de mejora.
- **No son:** para encontrar *soluciones* a las fallas, para *aprobar* un producto, ni para *evaluar a las personas*.

Roles

Rol	Responsabilidad
Autor	Crea o mantiene el producto; inicia el proceso eligiendo moderador; entrega el producto; reporta tiempo de retrabajo y total de defectos
Moderador	Planifica y lidera; elige roles con el autor; entrega el material a los inspectores con 48 h de anticipación; coordina la reunión evitando conductas inapropiadas; hace seguimiento de defectos
Lector	Lee el producto durante la reunión
Anotador	Registra los hallazgos
Inspector	Examina el producto antes de la reunión buscando defectos; registra su tiempo de preparación

Proceso de inspección (convencional)

1. **Planificación:** elegir equipo, preparar material y calendario.
2. **Visión general:** presentar el proceso y el producto.
3. **Preparación:** análisis individual de cada inspector para encontrar potenciales defectos.
4. **Reunión de inspección:** el equipo recolecta los defectos y filtra falsos positivos. Termina con una **decisión**: aceptar sin modificaciones, rechazar, o aceptar provisoriamente; y un **informe** (qué se revisó, quién, qué se encontró, conclusiones).
5. **Corrección:** el autor corrige los defectos.
6. **Seguimiento:** verificar correcciones y recolectar datos.

Tiempos de referencia (código / documentos): reunión de 2 h máximo; planificación 15 min / 30 min; vista previa 500 LOC/h / 500 líneas de texto/h; preparación 100 LOC/h / 140 líneas/h; inspección 125 LOC/h / 140 líneas/h; mejora del proceso 30 min / 45 min; máximo por inspección 250 LOC / 280 líneas de texto.

Métricas sugeridas

Métrica	Fórmula
Densidad de defectos	Total de defectos / tamaño
Total de defectos	Mayores + menores
Esfuerzo de la inspección	Planificación + preparación + reunión + retrabajo
Esfuerzo por defecto	Esfuerzo de inspección / total de defectos
% de reinspecciones	Reinspecciones / inspecciones
Defectos corregidos sobre total de defectos	Defectos corregidos / total de defectos (en la diapo esta fila trae la fórmula esfuerzo de inspección / tamaño actual, que parece un error)

Puntos clave

Revisar el producto, no al productor; fijar una agenda y cumplirla; limitar el debate; enunciar los problemas pero no intentar resolverlos en la reunión; tomar notas; limitar participantes e insistir en la preparación previa; usar checklists; dar recursos y entrenamiento; repasar revisiones anteriores.

9. Auditorías de software

Una auditoría de calidad de software es una **evaluación independiente** de productos o procesos para asegurar el cumplimiento de estándares, lineamientos, especificaciones y procedimientos, basada en criterios objetivos (IEEE 1028). Esos criterios incluyen documentación que especifique la forma o el contenido de los productos a desarrollar, el proceso con el que se desarrollan y cómo debería medirse el cumplimiento de estándares o lineamientos. Es un instrumento del aseguramiento de calidad: cuesta esfuerzo, pero sus beneficios son mayores.

Objetivos de SQA (contexto): controlar el software y el proceso; asegurar el cumplimiento de estándares y procedimientos; asegurar que los defectos de producto, proceso o estándares se informen a la gerencia.

¿Por qué auditar? Da una opinión objetiva e independiente; identifica áreas de insatisfacción potencial del cliente; asegura al cliente que cumplimos sus expectativas; identifica oportunidades de mejora.

Beneficios: evaluar el cumplimiento del proceso; determinar la implementación efectiva del proceso organizacional, del proceso del proyecto y de las actividades de soporte; dar visibilidad a la gerencia. Resultado: mejores productos → clientes satisfechos → crecimiento.

Tipos de auditoría

Tipo	Qué valida	Detalle
De proyecto	El **cumplimiento del proceso** de desarrollo	Según el **PACS** (Plan de Aseguramiento de Calidad de Software), que nombra al responsable. Verifica la consistencia del producto a medida que evoluciona: interfaces consistentes con la ERS, requerimientos de la ERS validados en el plan de V&V, diseño (DDS) que satisface la ERS, código consistente con el DDS. Incluye inspecciones y revisiones de diseño y prueba
De configuración funcional (FCA)	Que el producto cumpla sus requerimientos (se asocia con validación*)	Compara el software construido (ejecutables y documentación) con la ERS: el código implementa *sólo y completamente* lo especificado. QA valida que la **matriz de rastreabilidad** esté actualizada
De configuración física (PCA)	Que el ítem **tal como está construido** coincida con su documentación técnica (se asocia con verificación*)	Compara el código con la documentación de soporte a entregar. El software se entrega solo cuando se corrigen las desviaciones

Ojo: las diapos dicen que hay tres tipos de auditoría (de proyecto, de configuración funcional y de configuración física) y usan "valida" para los tres; asociar la funcional con validación y la física con verificación (*) es una interpretación. Lo que sigue no está en las diapos.

Auditorías al grupo de calidad (auditorías externas de aseguramiento de calidad): las hace personal externo a la organización para evaluar a los miembros de QA, es decir, verificar que sus evaluaciones dentro de la organización se estén haciendo correctamente.
Auditoría de proyecto en tradicional vs ágil: en tradicional el auditor es externo al proyecto (a veces a la empresa); en Scrum es el propio equipo el que revisa su proceso, en la Sprint Retrospective. (No está desarrollado en las diapositivas; sale del resumen de Joaquín.)

Roles y responsabilidades

Rol	Responsabilidades
Gerente de SQA	Prepara el plan de auditorías, calcula su costo, asigna recursos; responsable de resolver las no conformidades

Rol	Responsabilidades
Auditor	Acuerda fecha y comunica el alcance; recolecta y analiza evidencia objetiva relevante y suficiente; realiza la auditoría; prepara el reporte; hace seguimiento de los planes de acción
Auditado	Acuerda fecha; participa; aporta evidencia; contesta el reporte; propone el plan de acción para las deficiencias; comunica su cumplimiento

Proceso de auditoría

1. **Preparación y planificación:** el auditado solicita la auditoría; se acuerdan lugar, fecha, alcance, objetivo y participantes; el auditor puede pedir y revisar documentación preliminar; el auditado prepara la información.
2. **Ejecución:** *reunión de apertura* (presentar participantes, revisar cronograma, alcance y objetivo, informar el carácter y el proceso, aclarar dudas) y *ejecución propiamente dicha* (el auditor pregunta, pide y revisa evidencia, completa el checklist e informa la fecha del reporte).
3. **Análisis y reporte de resultados:** *evaluación* (revisar checklist, notas y documentación; verificación cruzada de desviaciones contra el proceso; informe preliminar), *reunión de cierre* (presentar y acordar resultados, fijar fecha del reporte final) y *entrega del reporte final*.
4. **Seguimiento:** el auditado analiza las desviaciones y envía un plan de acción con fechas; el auditor lo revisa y aprueba, sigue las acciones, verifica su cumplimiento y cierra las desviaciones.

Checklist: fecha, auditados (con su rol), auditor, proyecto, fase actual, objetivo y alcance, lista de preguntas (ej.: ¿existe un plan de proyecto actualizado?, ¿hay responsable por actividad?, ¿existe una ERS con requerimientos identificados unívocamente?, ¿los cambios siguieron el proceso de cambios?, ¿se asignaron recursos a las actividades de soporte?, ¿los planes están disponibles para todos los involucrados?, ¿está descripto cada requerimiento?). El auditor además debe asegurarse de que los planes estén basados en los requerimientos, las actividades planificadas se hayan realizado, todos los involucrados se hayan comprometido con la última versión de los planes, los cambios a los planes estén aprobados y su decisión documentada a tiempo, y los riesgos se hayan identificado y comunicado. En requerimientos, que estén revisados y aprobados por todos los involucrados y que, si cambiaron, hayan seguido el proceso de cambios y se hayan actualizado los planes.

Durante la auditoría el auditor debe: escuchar sin interrumpir, observar el lenguaje corporal (y manejar el propio), tomar notas, preguntar, repetir lo escuchado para confirmar. **Técnica de cuestionario:** empezar con preguntas abiertas (quién, qué, cómo, cuándo, dónde), preguntas cortas y puntuales, terminar con preguntas cerradas para clarificar.

Reacciones comunes del auditado: querer impresionar, ansiedad, sentirse examinado, usar la auditoría para quejarse de la empresa, dar demasiada información, enojo o nervios.

Técnicas y herramientas: checklist, muestreo, revisión de registros, herramientas automatizadas.

Reporte y resultados

Contenido del reporte: identificación, fecha, auditor, auditados, proyecto, fase, lista de resultados, comentarios, solicitud de planes de acción.

Resultado	Qué es	¿Plan de acción?
Buena práctica	Práctica mucho mejor de lo esperado (sistema efectivo, alto conocimiento y cooperación, práctica superior a cualquier otra vista)	No
Desviación	Disconformidad del producto con sus requerimientos, falta de control, o desvío del proceso que genera incertidumbre sobre la calidad	**Sí**
Observación	Condición que debería mejorarse o que puede derivar en una desviación futura	No

Métricas de auditoría: esfuerzo por auditoría, cantidad de desviaciones, duración, desviaciones clasificadas por área de proceso de CMMI.

10. Testing de software

La prueba de software en contexto

Está en el contexto del aseguramiento de calidad, pero no es control de calidad de proceso ni de producto: es un tema en sí mismo, bajo el paraguas de la **Administración de Configuración**.

¿Qué es el testing?

Es un proceso **destructivo** de tratar de encontrar defectos en el código, cuya presencia se asume: se va con una actitud negativa para demostrar que algo es incorrecto. **Un testing exitoso es el que encuentra defectos** (el objetivo no es asegurar que el software sea confiable). A nivel mundial representa entre el 30 y el 50% del costo de un software confiable.

Definiciones formales:

- **ISTQB:** proceso de todas las actividades del ciclo de vida, estáticas y dinámicas, de planificación, preparación y evaluación del software para determinar que satisface los requerimientos, demostrar que sirve a su propósito y detectar defectos.
- **IEEE 610:** operar un sistema bajo condiciones especificadas, observar o registrar los resultados y evaluar algún aspecto.
- **IEEE 829:** analizar un ítem de software para detectar las diferencias entre lo existente y lo requerido (bugs) y evaluar sus características.

Testing en el contexto: asegurar la calidad vs controlar la calidad

Una vez definidos los requerimientos de calidad, hay que tener en cuenta que:

- La calidad no puede "inyectarse" al final.
- La calidad del producto depende de tareas realizadas durante todo el proceso.
- Detectar errores en forma temprana ahorra esfuerzo, tiempo y recursos.
- La calidad abarca el producto y también el proceso y cómo mejorarlo, lo que evita defectos recurrentes.
- **El testing NO puede asegurar ni calidad en el software ni software de calidad.**

Error vs defecto

Error: se detecta en la misma etapa en que se introdujo (ej.: escribiendo código hago una revisión y lo encuentro). **Defecto:** un error no detectado que pasa a una etapa posterior. En el testing encontramos defectos. (Revisiones técnicas usa otro vocabulario: ver sección 8.)

Defectos: severidad y prioridad

- **Severidad** = gravedad del defecto: 1 Bloqueante (no puedo seguir con el caso de prueba), 2 Crítico (compromete el resultado), 3 Mayor, 4 Menor, 5 Cosmético (ortografía o visualización); es una clasificación tentativa.
- **Prioridad** = urgencia para resolverlo: 1 Urgencia, 2 Alta, 3 Media, 4 Baja. Depende del contexto y no siempre coincide con la severidad.

Niveles de prueba

Los niveles son como escalones: cada uno abarca más que el anterior. Unitario → integración → sistema → aceptación.

- **Unitario** (desarrollador): se prueba cada componente por separado, de forma independiente, tras construirlo. Se hace con acceso al código y con el apoyo del entorno de desarrollo (framework de pruebas unitarias como JUnit, herramientas de depuración). Los errores se reparan apenas se encuentran, sin constancia oficial. Son las más fáciles de automatizar; acá se encuentran más errores que defectos.
- **Integración** (tester): verifica que las partes que funcionan bien aisladas también lo hagan en conjunto. Tiene que ser incremental: top-down, bottom-up o, lo ideal, una combinación. Los módulos críticos se prueban lo antes posible; conectar de a poco las partes más complejas y minimizar la necesidad de programas auxiliares.
- **Sistema** (tester): la aplicación funcionando como un todo (construcción final). Determina si el sistema en su globalidad opera satisfactoriamente (recuperación de fallas, seguridad y protección, stress, performance) e investiga requerimientos funcionales

y no funcionales. El entorno de prueba tiene que parecerse lo más posible al de producción, para minimizar incidentes debidos al ambiente.

- **Aceptación** (usuario): la hace el usuario para determinar si la aplicación se ajusta a sus necesidades. La meta es establecer confianza en el sistema; encontrar defectos no es el foco. Comprende pruebas **alfa** (el usuario en ambiente de laboratorio) y **beta** (en ambientes de trabajo reales).

Ambientes para la construcción del software

Desarrollo → Prueba → Pre-producción → Producción. Ambiente y entorno son sinónimos. Las unitarias se hacen en desarrollo o prueba; las de integración, en prueba. Producción es donde el software opera; replicar sus características (performance, concurrencia, seguridad, redundancia) es caro, por eso las pruebas de sistema y de aceptación se hacen en pre-producción, lo más parecido posible a producción. Excepción: si el software es nuevo y todavía no está en operación, se puede usar producción para las pruebas beta; una vez que se libera la primera versión, ya no se prueba ahí.

Caso de prueba

Conjunto de condiciones o variables bajo las cuales el tester determina si el software funciona correctamente o no. Una buena definición permite **reproducir** defectos (clave para que, después de la corrección, el software sea confiable). La diapo lo introduce con el ejemplo de explicarle a un compañero cómo encontrar a Wally: pasos concretos que llevan siempre al mismo lugar.

Objetivo, criterio y restricción: descubrir errores, en forma completa, con el mínimo esfuerzo y tiempo. "Los bugs se esconden en las esquinas y se congregan en los límites" (Boris Beizer). No se pueden definir casos infinitos: hay que elegir la menor cantidad que cubra lo más posible.

Derivación de casos: desde documentos del cliente, información de relevamiento, requerimientos, especificaciones de programación o el código. Cuanta más documentación, más fácil armar los casos; si los requerimientos son generales o poco precisos, los casos tienen que ser más detallados.

Conclusiones sobre la generación de casos: ninguna técnica es completa; atacan distintos problemas, así que lo mejor es combinarlas; depende del código a testear; sin requerimientos todo es mucho más difícil; tener en cuenta la conjetura de defectos; ser sistemático y documentar los supuestos.

Condición de prueba: la reacción esperada de un sistema frente a un estímulo particular (las distintas entradas). Es el contexto del sistema a tener en cuenta para la prueba. Cada condición se prueba con al menos un caso.

Partes de un caso de prueba (la unidad de la actividad de prueba): **objetivo** (la característica a comprobar), **datos de entrada y de ambiente** (qué variables y qué valores concretos se usan en cada paso) y **comportamiento esperado** (la salida o acción esperada según los requerimientos).

Estrategias y métodos de prueba

Caja negra y caja blanca: está todo junto en la sección 11, para no repetirlo acá.

Ciclo de test y regresión

Ciclo de test: la ejecución de la totalidad de los casos de prueba sobre una misma versión del sistema. Se ejecuta el ciclo 1, se corrigen los defectos y se corre otro ciclo sobre la nueva versión, hasta cumplir el criterio de aceptación acordado con el cliente (ej.: hasta 3 ciclos, o hasta que no queden defectos bloqueantes ni críticos).

Regresión: al terminar un ciclo y reemplazar la versión, se hace una verificación total de la nueva versión, para prevenir defectos nuevos introducidos al corregir los detectados. No alcanza con repetir los casos que fallaron: al corregir un defecto suelen introducirse dos o tres nuevos. Sin regresión, un caso que antes pasaba puede esconder un problema nuevo o un error escondido; con regresión se detecta y se arregla.

Proceso de prueba

Flujo: planificación → identificación y especificación → ejecución → análisis de fallas (si hay defectos, se vuelve a ejecutar) → ¿momento de dejar de probar? → fin de las pruebas.

- **Planificación y control:** entender metas y objetivos del cliente, stakeholders, proyecto y riesgos. Se arma el **Test Plan** (riesgos y objetivos, estrategia, recursos, criterio de aceptación). Controlar = revisar resultados, cobertura y criterio de aceptación, y tomar decisiones.
- **Identificación y especificación:** revisar la base de pruebas, verificar las especificaciones, evaluar la testeabilidad de requerimientos y sistema, identificar los datos, diseñar y priorizar los casos, diseñar el entorno.
- **Ejecución:** priorizar los casos, crear los datos de prueba, automatizar lo necesario, armar conjuntos de casos, implementar y verificar el ambiente, ejecutar, registrar resultados y versiones, comparar real vs esperado.
- **Evaluación y reporte:** evaluar los criterios de aceptación, reportar a los stakeholders, consolidar la información, verificar entregables y corrección de defectos, lecciones aprendidas.

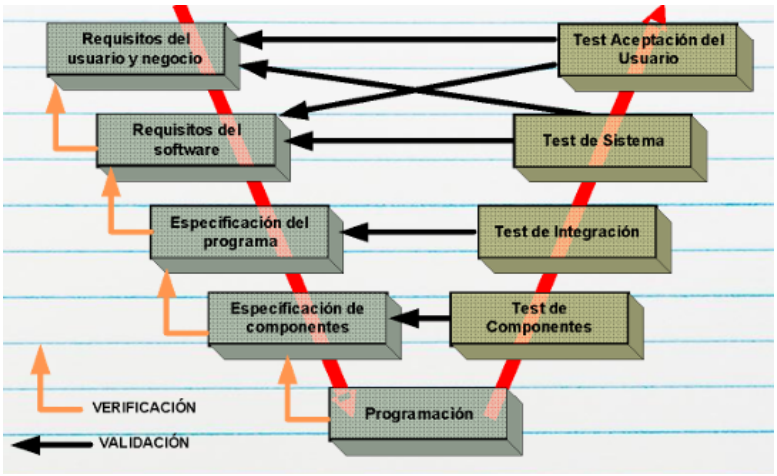
La planificación (plan de prueba) es la parte más estratégica y es propia del enfoque tradicional: en Scrum no hay plan de prueba, pero sí se prueba; identificar/especificar y ejecutar casos (automatizados o no) se hace en ambos. Si el resultado no es el esperado, el testing cumplió su objetivo: se analiza la falla para corregirla, y ejecución y análisis se repiten hasta cumplir el criterio de aceptación, que marca el fin de las pruebas.

El testing y el ciclo de vida

Verificación: ¿estamos construyendo el sistema correctamente?
Validación: ¿estamos construyendo el sistema correcto? (ver también sección 8).

Objetivos de involucrar el testing temprano: dar visibilidad temprana al equipo de cómo se va a probar el producto y disminuir los costos de corregir defectos. Se puede empezar desde el principio: con los requerimientos definidos ya se puede trabajar, sin esperar el código.

Modelo en V (bajando por la izquierda, programación en la base, subiendo por la derecha): requisitos del usuario y del negocio ↔ test de aceptación del usuario; requisitos del software ↔ test de sistema; especificación del programa ↔ test de integración; especificación de componentes ↔ test de componentes. Cada nivel de prueba verifica/valida lo definido en su etapa.



Romper mitos

- "El testing es una etapa que comienza al terminar de codificar" → falso: empieza desde los requerimientos.
- "El testing es probar que el software funciona" → falso: busca encontrar defectos.
- "Testing = calidad de producto" y "testing = calidad de proceso" → falso: ayuda a que el software sea confiable, pero no es calidad.
- "El tester es el enemigo del programador" → falso: tienen objetivos contrapuestos, pero el tester solo hace visible lo que pasa.

¿Cuánto testing es suficiente?

El testing exhaustivo es imposible. Cuánto probar depende del **riesgo** (qué probar primero, a qué dedicarle más esfuerzo, qué no probar por ahora) y de los **costos** del proyecto. Para decidir que una fase de testing terminó se usa el **criterio de aceptación**, acordado con el cliente. Puede ser definido en terminos de: costos, % de tests corridos sin fallas, fallas predichas que aún quedan, o que no haya defectos de cierta severidad.

Principios del testing

- El testing muestra la **presencia** de defectos.
- El testing exhaustivo es imposible.
- Testing temprano.
- Agrupamiento de defectos (se concentran en pocos módulos).
- Paradoja del pesticida (repetir los mismos casos deja de encontrar defectos nuevos; pasa sobre todo si están automatizados: corren sin fallas pero los defectos quedan ocultos).
- El testing depende del contexto.

- Falacia de la ausencia de errores (que no haya errores no implica que el sistema sirva al usuario).

Además: un programador debería evitar probar su propio código (salvo las pruebas unitarias), y una unidad de programación no debería probar sus propios desarrollos. Probar que no hace lo que debe es la mitad de la batalla; la otra mitad es ver qué hace lo que no se supone que debería hacer (esa actitud negativa es lo más difícil). No planificar el esfuerzo de testing suponiendo que no habrá defectos: que los haya es lo esperable, así que el tester no viene a criticar al programador.

La psicología del testing

Buscar fallas puede verse como una crítica al producto o a su autor, y construir software requiere otra mentalidad que testearlo (desarrolladores vs testers).

Smoke test

Primera corrida de los tests de sistema, que da cierta seguridad de que el software no provoca una falla catastrófica. Se hace con la primera versión: rápida y superficial, sin caminos alternativos ni casos rebuscados. El nombre viene de los ingenieros electrónicos, que probaban si una placa empezaba a echar humo al aplicarle corriente. No es lo mismo que el sanity test, que corre los tests para asegurar que las funcionalidades básicas no tengan fallas críticas.

Tipos de prueba

- **Funcional:** se basa en las funciones y características (descriptas en los documentos o entendidas por los testers) y su interoperabilidad con sistemas específicos; puede basarse en requerimientos o en procesos de negocio. Es el tipo de prueba más fácil.
- **No funcional:** prueba **cómo** funciona el sistema. ¡No hay que olvidarlas! Los requerimientos no funcionales son tan importantes como los funcionales: performance, carga, stress, volumen, usabilidad, mantenimiento, fiabilidad, portabilidad, instalación. Acá es clave que el ambiente de prueba se parezca al de producción.
- **Interfaces de usuario:** las GUI son más complejas que las interfaces de caracteres: el usuario tiene el control y hay muchas combinaciones, así que requieren más pruebas.
- **Performance:** tiempo de respuesta y concurrencia.
- **Configuración:** cómo trabaja el sistema con cierta configuración: compatibilidad con drivers de video, conexiones de red, software de terceros, sistemas operativos, base de datos y middleware.

11. Estrategias y métodos de prueba: caja negra y caja blanca

Estrategias y métodos

Hay dos **estrategias**: **caja blanca** y **caja negra**. Son complementarias: las dos buscan abarcar la mayor cobertura de pruebas con el menor esfuerzo.

¿Para qué usar métodos? El tiempo y el presupuesto son limitados, así que hay que pasar por la mayor cantidad de funcionalidades con la menor cantidad de pruebas. No se pueden definir casos de prueba de forma infinita: hace falta un mecanismo para definir la menor cantidad de casos que cubra lo más posible.

Caja negra

Se llama así porque se definen las entradas y el resultado esperado, pero no se ve cómo llega el sistema a ese resultado: lo que pasa en el medio no se conoce. Se divide en dos grupos:

- **Basadas en especificaciones:** partición de equivalencias, análisis de valores límite, etc.
- **Basadas en la experiencia:** adivinanza de defectos y testing exploratorio.

Partición de equivalencias

Se divide el dominio de entrada en clases. Una **clase de equivalencia** es un grupo de valores que el sistema trata igual, así que alcanza con probar un representante de cada clase. Dos valores van en clases distintas si son "significativamente diferentes": disparan otro flujo, otro mensaje de error, cambian la pantalla u otras opciones, o activan otra regla de negocio. Tiene **dos pasos**:

- **Identificar las clases de equivalencia (válidas y no válidas).** Según el tipo de condición de entrada: rango de valores continuos, valores discretos, selección simple o selección múltiple.
- **Identificar los casos de prueba.** Cubrir todas las clases válidas con la menor cantidad de casos, y una sola clase no válida por caso (para que un error no tape a otro).

Análisis de valores límite

Es una variante de la partición de equivalencias: en vez de elegir cualquier elemento como representante de una clase, se eligen los **bordes** de cada clase. En las esquinas o bordes se congregan los defectos ("los bugs se esconden en las esquinas y se congregan en los límites", Beizer): hay que probar los límites de lo tolerable.

Caja negra basada en la experiencia

Es el testing del tester experimentado que, sin saber bien por qué, sabe adónde ir a buscar el defecto. No hay nada establecido de antemano.

- **Adivinanza de defectos**: el tester usa su experiencia para anticipar dónde suele haber errores (vacíos, ceros, nulos, límites, formatos raros).
- **Testing exploratorio**: se diseña y ejecuta al mismo tiempo, aprendiendo del sistema mientras se prueba, sin casos escritos de antemano.

Caja blanca

Se basa en el análisis de la **estructura interna** del software o de un componente. Se mira la estructura del código para ver si en algún lugar el sistema puede dar algo no esperado. Con caja blanca **se puede garantizar el testing coverage**. Métodos: cobertura de enunciados o caminos básicos, de sentencias, de decisión, de condición, de decisión/condición, múltiple, etc.

Elegir un método

Cada método tiene fortalezas y debilidades: puede ser bueno para algunas cosas y no para otras. **El mejor método es no usar un único método**: usar una variedad de técnicas ayuda a un testing efectivo. Ninguna técnica es completa, porque cada una ataca problemas distintos; sin requerimientos, todo es mucho más difícil.

Para el práctico del parcial se diseñan casos de caja negra: partición de equivalencias + valores límite. Para no confundir: los **niveles** (unitaria, integración, sistema, aceptación) dicen qué tan grande es lo que se prueba; los **tipos** (funcional, no funcional, GUI) dicen qué aspecto se prueba; las **estrategias y técnicas** dicen cómo se diseñan los casos.

12. Testing en ambientes ágiles

En ágil el testing deja de ser una fase al final y pasa a ser una actividad continua de todo el equipo, que guía el desarrollo.

Testing Manifesto (valoramos más lo de la izquierda)

Surge alineado con el manifiesto ágil pero aplicado al testing, y es disruptivo respecto del testing tradicional (proceso destructivo, actitud negativa). Es la base sobre la que se construye el resto de los temas.

- Pruebas **durante** sobre pruebas al final. No es un proceso en cascada: se prueba mientras se construye.
- **Prevenir** defectos sobre encontrar defectos.
- **Entender** lo que se está probando sobre verificar la funcionalidad. Todo lo que se pueda chequear como un checklist debería automatizarse; lo manual se reserva para donde agrega valor.
- **Construir** un mejor sistema sobre romper el sistema.
- **Responsabilidad de todo el equipo** por la calidad sobre "el tester es responsable de la calidad".

Tradicional (cascada) vs ágil: en cascada el testing es una etapa al final (después del desarrollo) que busca la conformidad estricta con una especificación previa, a cargo de testers, con mucha documentación y feedback tardío. En ágil se busca entregar valor continuo: se prueba durante toda la iteración (sin esperar a terminar el diseño o el código), todo el equipo, con automatización y feedback inmediato.

Ciclo de vida del testing vs ciclo ágil (diapo): el ciclo de vida tradicional del testing es una secuencia propia (identificar requerimientos, criterios de aceptación, estrategia, planificación y diseño, ejecución, seguimiento de defectos y corrección, revisión y auditoría,

aceptación y línea base). En ágil el testing es una parte más de un ciclo corto que va del backlog al despliegue (definir, analizar, diseñar, probar), y dentro de cada iteración la codificación, la corrección de bugs y el testing ocurren en paralelo.

En ágil (iterativo e incremental) cada incremento se prueba apenas se termina: **los programadores nunca se adelantan a los testers, porque una historia no está "done" hasta que se probó**. La mayor diferencia para el tester es el **feedback rápido**, y no hay "guardianes" que frenen el avance en hitos.

9 principios del testing ágil

1. **El testing se mueve hacia adelante en el proyecto:** se adelanta y empieza desde el inicio (con las historias), en vez de esperar al final.
2. **El testing no es una fase:** es parte del proceso y se hace a lo largo de toda la iteración.
3. **Todos hacen testing:** en línea con el manifiesto, todo el equipo es responsable de la calidad, no solo el tester.
4. **Reducir la latencia del feedback:** que pase el menor tiempo posible entre que se introduce un defecto y se detecta (por eso se automatiza).
5. **Las pruebas representan expectativas:** expresan lo que espera el usuario; son la forma de especificar el comportamiento esperado.
6. **Mantener el código limpio:** corregir los defectos rápido, en la misma iteración, sin acumularlos.
7. **Reducir la sobrecarga de documentación de las pruebas:** documentar solo lo que agrega valor; las pruebas automatizadas sirven como documentación.
8. **Las pruebas son parte del "done":** una historia no está terminada si no está probada.
9. **De probar al final a desarrollo conducido por pruebas:** las pruebas guían el desarrollo (TDD, ATDD).

Los principios no dicen cómo hacerlo: son lineamientos y la base para institucionalizar el cambio. Las prácticas son lo concreto que se implementa en el equipo para materializarlos (lo más fácil de implementar, porque son herramientas).

6 prácticas concretas

1. Pruebas de unidad e integración automatizadas. Se automatizan para reducir la latencia del feedback (no todas las pruebas se pueden automatizar).
2. TDD (Test Driven Development). Las pruebas se escriben antes que el código.
3. Pruebas de regresión a nivel de sistema automatizadas. Después de corregir un defecto se prueba todo de nuevo, porque el arreglo puede romper otras partes.
4. ATDD (desarrollo conducido por pruebas de aceptación). En ágil las pruebas de aceptación las define el PO; es más amplio que TDD.
5. Pruebas exploratorias. Por lo general no se pueden automatizar.
6. Control de versiones de las pruebas junto con el código.

ATDD: primero la feature: antes de desarrollar una historia, PO, desarrolladores y testers acuerdan **pruebas de aceptación** de alto nivel (con ejemplos concretos); se automatizan y recién ahí se programa la solución. Esas pruebas definen cuándo la historia está terminada. TDD trabaja a nivel de unidad (construir bien); ATDD a nivel de negocio (construir lo correcto). **Ciclo de la diapo:** la feature (con ejemplos) se baja a historias de usuario → se exploran ejemplos y se escriben pruebas de aceptación de alto nivel → más pruebas → automatización de las pruebas → programar y ejecutar las pruebas → explorar (pruebas exploratorias, y otras pruebas) → aceptación de la historia. La corrección de defectos está en el centro del ciclo: se corrige en cualquier paso donde aparezca un defecto.

Roles y competencias del tester

En ambientes tradicionales: el tester suele estar en un equipo de QA independiente del desarrollo; entra al final, con el código terminado, a encontrar defectos contra la especificación.

En ambientes ágiles el tester es parte del equipo desde el inicio y tiene un rol central:

- **Con los developers**: definición de estrategias de prueba, automatización de pruebas funcionales (ATDD), detecta y notifica bugs, feedback temprano de las historias, sugiere mejoras funcionales y de usabilidad.
- **Con el Product Owner (le ayuda en):** preparación y revisión de historias, definición de criterios de aceptación, mejor entendimiento del dominio de negocio, identificar dependencias.
- **Con el System Team**: prepara la documentación de las historias a integrar y probar, difunde el plan, los casos y la data de prueba para la integración, ayuda a conocer la completitud del feature, recibe bugs y los comunica al PO y a los devs.

- **Tareas propias:** preparación y mantenimiento del ambiente de pruebas, casos y data de prueba de las historias del sprint, validación del "Done", registro y notificación de bugs, aceptación de la historia junto al PO, métricas de calidad.

Agrega mucho más valor que alguien de QA en un ambiente tradicional. La calidad es de todo el equipo (whole-team approach): cualquiera puede tomar tareas de testing.

Desafíos culturales

Pasar a testing ágil es sobre todo un cambio cultural: lo más difícil es institucionalizar los valores y principios. La diapo lo divide en **5 aspectos**:

1. **Cultura organizacional:** filosofía de calidad (si la calidad es de todos o de un área de QA), ritmo sostenible, relación con el cliente, tamaño de la organización y empoderar al equipo.
2. **Barreras para el éxito:** pérdida de identidad del tester, roles adicionales, falta de capacitación, no entender los conceptos ágiles, experiencias pasadas y diferencias culturales entre roles.
3. **Introducir el cambio:** hablar de los miedos, darle al equipo la propiedad del cambio y celebrar los éxitos.
4. **Expectativas de la gerencia:** el cambio cultural también es para los managers (entender que la calidad es de todo el equipo y que cambian las métricas y los reportes), y hay que hablarles en su idioma (costos, tiempos, riesgos).
5. **El cambio no es fácil:** tener paciencia, dejar que sientan el dolor (que vean el problema por sí mismos), construir credibilidad, desarrollo profesional, cuidarse de la mentalidad de "policía de la calidad" y, si nada cambia, votar con los pies (irse).

Cuadrantes del testing ágil (Marick / Crispin & Gregory)

	Pruebas que guían el desarrollo	Pruebas que critican el producto
Orientadas al negocio	**Q2** — ejemplos, pruebas de aceptación de la historia, pruebas de experiencia de usuario (UX), prototipos, simulaciones (automatizadas y manuales)	**Q3** — pruebas exploratorias, flujos de trabajo, usabilidad, aceptación (UAT), monitoreo y observabilidad (manuales)
Orientadas a la tecnología	**Q1** — pruebas unitarias y de componentes, a nivel de código (automatizadas)	**Q4** — rendimiento, carga, seguridad, recuperabilidad y otros atributos de calidad ("…ilidad") (herramientas)

- **Para qué sirven:** muestran los tipos de testing disponibles, a qué apunta cada uno y cuáles conviene automatizar o hacer a mano; son una taxonomía para que se contemplen todas las dimensiones del testing. La **numeración no indica el orden** en que se hacen; depende del riesgo y del proyecto. Automatización: Q1 y Q4 (de cara a la tecnología) se sugiere automatizarlos; Q2 combina pruebas automatizadas y manuales; Q3 no se puede automatizar.
- **Apoyan al equipo (Q1 y Q2):** guían el desarrollo y, automatizadas, son la red de seguridad para refactorizar. Q1 mide la **calidad interna** (la definen los programadores, no se negocia con el cliente; su propósito principal es TDD; se automatiza junto con la integración continua, con feedback en minutos). Q2 define la **calidad externa** con ejemplos del cliente, en su lenguaje de negocio.
- **Critican el producto (Q3 y Q4):** "criticar" no es negativo, es evaluar para mejorar. Q3 es prueba manual que solo una persona puede hacer, emulando al usuario real (el exploratorio es central y ahí aparecen los bugs más serios). Q4 evalúa performance, robustez y seguridad con herramientas que simulen la situación a probar (ej. carga o estrés).
- Las **pruebas de aceptación** no son solo Q2: abarcan también Q3 y Q4 (usabilidad, performance).

Pirámide del testing ágil

- **Base: pruebas unitarias / de componentes**, realizadas por el programador. Son la mayoría: las más rápidas, baratas y de feedback más rápido; se intenta llevar a este nivel todo lo posible.
- **Medio: capa de servicio / API:** reglas de negocio y pruebas funcionales, sin pasar por la interfaz ("detrás de la GUI").
- **Punta: pruebas de flujo de trabajo a través de la interfaz gráfica.** Las menos: más caras, lentas y frágiles (cambiar un elemento de la pantalla las rompe).
- **Nube: pruebas manuales / exploratorias.** La diapo "Entendiendo la pirámide del testing ágil" la muestra con otras capas (de abajo hacia arriba: unitarias, de aceptación, de sistema y restricciones, exploratorias y "otras") y agrega para qué sirven: guiar el

desarrollo, aceptación y regresión, encontrar huecos, y pruebas que no es práctico automatizar. Las capas de abajo se ejecutan automáticamente y las de arriba, manualmente.

Pirámide del testing: Tradicional vs ágil

Tradicional (para buscar bugs): pirámide invertida: GUI end-to-end 80–90%, integración 5–15%, unitarias 1–5%. **Ágil (para prevenir bugs):** unitarias 80–90%, aceptación 5–15%, GUI 1–5%. La diferencia de fondo: el tradicional apunta a encontrar defectos; el ágil, a prevenirlos.

Niveles de prueba: el dibujo muestra una aplicación en capas (browser, web, negocio, datos y base de datos): las **unitarias** prueban cada componente por separado; las de **integración**, cómo se comunican componentes de distintas capas; las de **sistema**, la aplicación completa de punta a punta; y las de **aceptación**, ese mismo sistema desde el punto de vista del usuario.

Beneficios de la automatización

Rápida ejecución, reusabilidad, mayor cobertura, mayor precisión, mayor alcance, reducción de costos. La reusabilidad está ligada a la regresión; automatizar libera a las personas para hacer pruebas exploratorias y es la base para entregar rápido sin acumular deuda técnica.

13. TDD – Test Driven Development

"El acto de diseñar tests es uno de los mecanismos conocidos más efectivos para prevenir errores… El proceso mental que debe desarrollarse para crear tests útiles puede descubrir y eliminar problemas en todas las etapas del desarrollo" (B. Beizer). TDD (Test-Driven Development) viene de **Kent Beck**, en **XP**.

Desarrollo tradicional vs TDD

- **Tradicional:** diseñar → desarrollar → probar. Los desarrolladores diseñan las pruebas unitarias **después** de escribir el código; el aseguramiento y control de calidad es más **reactivo**.
- **TDD:** diseñar → probar → desarrollar. Los desarrolladores diseñan las pruebas unitarias **antes** de escribir el código; el aseguramiento y control de calidad es más **proactivo**.

Desarrollo de software ágil con TDD

TDD dentro de ATDD: se crea una historia de usuario, se definen escenarios y, para cada escenario, se crea una prueba de aceptación que falla y se la hace pasar. Adentro de eso está el ciclo de TDD a nivel unitario (crear una prueba unitaria que falla → hacerla pasar → refactorizar). Al final se demuestra software funcionando y se vuelve a empezar con la siguiente historia.

¿Qué es TDD?

Desarrollo guiado por pruebas. Es una **técnica avanzada** que involucra otras dos prácticas: **escribir las pruebas primero** (Test First Development) y **refactorización** (Refactoring). Para escribir las pruebas generalmente se usan **pruebas unitarias**. Es diseño conducido por casos de prueba: primero escribo el caso de prueba y después el código para que ese caso sea exitoso. Son pruebas unitarias automatizadas.

Beneficios

Código robusto; más predecible y limpio (permite saber cuándo está terminado); más tolerante al cambio (con las pruebas escritas es más fácil de entender); más seguro; reduce la duplicación (si el test pasa, el código ya existe); más barato de mantener (más entendible); con la práctica acelera el desarrollo y habilita el despliegue continuo; asegura cobertura de prueba (código de calidad); obliga a pensar cómo se usa el componente (diseño de las API); promueve alta cohesión y bajo acoplamiento.

El corazón de TDD: ciclo Red – Green – Refactor

Red: el test falla. **Green:** el test pasa. **Refactor:** código y tests limpios. Primero elijo el requerimiento que voy a probar, escribo la prueba y después la implementación para que el test pase; una vez que la prueba funciona, refactorizo.

Las 3 leyes de TDD (Robert C. Martin)

- No escribir una línea de código hasta haber escrito un test que falla.

- No escribir más tests de los necesarios para que falle: normalmente con un test que falle alcanza.
- No escribir más código de producción que el necesario para que el test pase.

Pasos de TDD

Escribir un test → hacer que compile → hacer que falle (**Red**) → hacer un cambio para que pase → correrlo y que pase (**Green**) → quitar duplicaciones (**Refactor**).

Patrones

Son buenas prácticas o "recetas" ya probadas para cada momento del ciclo: los **red bar** ayudan a escribir el próximo test que falla, los **green bar** a hacerlo pasar rápido y los **de TDD en general** a mantener los tests ordenados y prolijos.

Grupo	Patrón	Qué hace
Red bar	One Step Test	Un test que verifica una pieza mínima de funcionalidad
	Starter Test	Primer test para explorar la API o el comportamiento
	Explanation Test	Comentarios o preguntas para aclarar qué se espera
	Learning Test	Conocer APIs externas cuyo comportamiento no sabemos
	Another Test	Cubrir un caso o variante adicional
	Regression Test	Reproducir un bug encontrado para que no vuelva
Green bar	Fake It	Devolver un valor fijo solo para que pase
	Triangulación	Un segundo test con otros datos fuerza una implementación general
	Obvious Implementation	Si la solución es trivial, implementarla directo
	One to Many	Resolver tests sobre colecciones
TDD	Test / Isolated Test / Test List / Test First / Assert First / Test Data / Evident Data	Nombrar con "test" + clase; tests independientes; escribir primero la lista de tests; test antes que código; escribir primero el assert; datos fáciles de leer; resultados esperados evidentes dentro del test. Otros patrones: de testing, de diseño y xUnit

Refactoring

Es una forma disciplinada de introducir cambios reduciendo la posibilidad de introducir defectos. Es una transformación del software que **preserva la estructura externa** y **mejora la estructura interna**. Son cambios que no modifican el comportamiento observable (todas las pruebas siguen pasando), eliminan la duplicación o la complejidad innecesaria, mejoran la calidad, hacen el código más simple y fácil de entender, lo flexibilizan y lo hacen más fácil de cambiar. Voy escribiendo el código y después lo tiro, porque lo voy mejorando.

¿Por qué es necesario? Evitar el "deterioro del diseño", limpiar el desorden, simplificar el código, aumentar la legibilidad y la comprensión, encontrar errores, reducir el tiempo de depuración e incorporar lo que se aprende sobre la aplicación. Rehacer las cosas es fundamental en todo proceso creativo.

¿Cómo se hace? Asegurarse de que las pruebas pasen (TDD) → encontrar código que "huela mal" → determinar cómo simplificarlo → hacer las simplificaciones → ejecutar las pruebas para asegurarse de que sigan pasando.

En resumen

TDD combina dos prácticas (test-first development y refactoring) en un ciclo de 4 pasos: la prueba debe fallar (rojo) → escribir el código mínimo para que pase → la prueba debe pasar (verde) → refactorizar para mejorar el código.